Engineering the Agentic Enterprise: A Curriculum Architecture for AI-Accelerated Software Development

Executive Diagnosis of the Baseline Curriculum

The 2024–2025 era of artificial intelligence in software engineering was defined by the rapid emergence of "vibe coding"—a paradigm characterized by unstructured, conversational prompting, heavy reliance on generative AI for raw code creation, and a purely reactive approach to debugging.[1] While this approach yielded rapid prototypes and impressive demonstrations, it fundamentally collapsed under the rigorous demands of enterprise production environments. Unchecked agentic generation introduced severe "comprehension debt," architectural fragmentation, and critical security vulnerabilities across the software supply chain.[2]

An analysis of the current "Foundations" baseline reveals a curriculum that remains trapped in this transitional phase. The existing structure over-weights abstract model mechanics—such as the Transformer mental model and token embeddings—and severely under-weights the operational realities of orchestrating AI agents in complex, multi-system enterprise environments.[4] The current design treats artificial intelligence as a sophisticated autocomplete tool rather than an autonomous execution engine that requires strict governance, specification-driven constraints, and rigorous reviewability.[6]

The strongest elements of the current baseline include the recognition of context windows as a limiting factor, the emphasis on prompt injection vectors, and the conceptual distinction between chatbot interfaces and raw model calls.[8] The progression toward a Capstone Pull Request (PR) remains a solid pedagogical anchor. However, raw Large Language Model (LLM) theory is vastly over-represented. Enterprise engineers do not need to construct transformers from scratch; they need to understand how context rot, token limits, and probabilistic outputs directly impact system design and reliability.[5] Theoretical prompt engineering—such as manual "Prompt Contracts"—is similarly outdated, as modern command-line interface (CLI) tools rely on deterministic scaffolding, configuration files, and predefined skills rather than manual prompt crafting.[10]

Critical gaps exist in the current syllabus. The baseline entirely misses the paradigm shift toward agentic orchestration.[12] It lacks instruction on Spec-Driven Development (SDD) [14], task decomposition for parallel agent teams [16], the Model Context Protocol (MCP) as the definitive enterprise standard for tool integration [17], and the critical operational distinction between read-only exploration and autonomous execution.[19] Furthermore, security instruction must evolve beyond basic prompt injection to encompass Zero Trust architectures for autonomous

agents, Identity and Access Management (IAM) for non-human actors, and the OWASP Top 10 for Agentic Applications.[21]

Regarding terminology, the industry is coalescing around the term "Agentic Engineering" to describe the disciplined orchestration of AI agents, drawing a hard line against the amateurism of "vibe coding".[12] However, for enterprise procurement and conservative engineering cultures, the term "agentic" can still trigger anxieties regarding unchecked autonomy and compliance risks.[24] Therefore, the umbrella course title is best positioned as "Structured AI-Assisted Engineering" or "AI-Accelerated Software Development." This naming convention satisfies enterprise risk management while allowing the internal curriculum to aggressively teach advanced agentic engineering patterns, bridging the gap between executive reassurance and cutting-edge practitioner reality.[26]

The Evolution to Structured Agentic Engineering

Category	Vibe Coding (Legacy)	Agentic Engineering (2026 Target)
Primary Interface	— Conversational flow via external chatbots or naive LLM prompting. — Heavy reliance on copy-pasting code with minimal syntax awareness.	✓ In-IDE orchestration of autonomous coding agents (e.g., Cursor, Claude Code). ✓ Agents read codebases natively, execute shell commands, and loop on failure.
Workflow Trigger	— High-level, unconstrained prompting driving a "creative flow." — Ideas prioritized over rigorous structure; ideal for prototypes or hackathons.	✓ **Spec-Driven Development:** intent → constraints → acceptance criteria. ✓ Uses small, verifiable prompts to trigger methodical, self-directed generation.
Constraint Mechanism	— Model generates without limitations ("just enough rope to hang themselves"). — Lacks boundaries, leading to arbitrary network calls or unchecked data mutations.	✓ Enforces explicit functional boundaries, restricting unbounded access. ✓ Ensures architectural coherence via consistent module patterns and interfaces.
Quality Validation	— Minimal human oversight results in blind trust of AI outputs. — Creates "trust debt," forcing engineers to reverse-engineer undocumented logic.	✓ Powered by automated review pipelines and domain-specific linters. ✓ Requires strict cross-model validation and structured human-in-the-loop approvals.
Security Posture	— Often yields brittle, opaque codebases prone to performance meltdowns. — High risk of critical vulnerabilities (e.g., inverted truthy permission checks).	✓ Strong security posture with robust accountability for enterprise readiness. ✓ Built-in audit trails, precise access controls, and strict change management.

Enterprise environments require a transition from unstructured, prompt-driven 'vibe coding' to rigorous, specification-driven 'agentic engineering' to ensure maintainability and security.

Data sources: Medium (Addy Osmani), Tateeda, Turing College, Reddit (r/AI_Agents)

Redesign Strategy

The redesign strategy for the Foundations phase pivots the curriculum from a theoretical exploration of AI models to an operational bootcamp in the Agentic Software Development Life Cycle (AISDLC).[28] The core objective is to transition learners from "operators" who write syntax to "orchestrators" who define architecture, manage context, enforce constraints, and review deterministic outputs.[29] Raw model theory is aggressively compressed. Only the mechanics that directly impact daily engineering—such as token budgeting, context rot, probabilistic variance, and context window management—are retained.[31] The goal is to build an engineer's intuition for model limitations and economic scaling, avoiding unnecessary academic exercises in neural network construction.

The Claude Code Command Line Interface (CLI) serves as the central nervous system of this redesign. It is not treated as a supplementary topic or a standalone tool; rather, it functions as the primary environment through which all engineering concepts are taught.[32] By standardizing on Claude Code, the curriculum deeply explores advanced, enterprise-safe features. Learners will master the configuration hierarchy via CLAUDE.md, deploy the Model Context Protocol (MCP) for secure internal tool integration, utilize deterministic hook systems for automated compliance, and invoke specialized agent teams for parallel task execution.[33] Every session requires terminal-based reasoning, ensuring the CLI becomes a natural, required extension of the enterprise engineer's workspace.

To actively combat the liabilities of "vibe coding," the curriculum formally introduces Spec-Driven Development (SDD).[14] Learners are trained to never allow an autonomous agent to write code without a preceding, version-controlled specification. The curriculum structurally enforces a strict workflow: codebase reconnaissance, specification drafting, read-only plan generation, human approval, and finally, execution.[20] This sequence teaches engineers to utilize AI for rigorous architectural mapping and requirement formulation before granting any write permissions to the file system. By enforcing atomic task decomposition and test-driven agentic loops, the course guarantees that all AI output is reviewable, testable, and strictly aligned with initial architectural intent.[37]

Security, governance, and auditability are elevated from a single module to a continuous, foundational thread, culminating in a dedicated focus on enterprise safety protocols. The curriculum integrates the OWASP Top 10 for Agentic Applications [21] and NIST Artificial Intelligence Risk Management Framework (AI RMF) guidelines.[39] Instruction zeroes in on "execution plane" risks, such as indirect prompt injection via compromised MCP servers, unauthorized tool escalation, and the absolute necessity of immutable audit logs for all agent actions.[22] Learners will actively configure Zero Trust boundaries within the CLI, applying strict least-privilege principles to specialized AI subagents.

Finally, the Foundations phase is meticulously engineered to bridge directly into the Labs and Applied phases. Instead of concluding with generic theoretical scenarios or isolated capstone

projects, the final module operates entirely as applied laboratory preparation. Learners will leverage the Claude Code CLI to perform legacy codebase reconnaissance, generate comprehensive test coverage plans, and execute continuous integration triage loops.[28] Graduates exit Foundations not just with a modern mental model of artificial intelligence, but equipped with a fully configured, secure CLI environment and the disciplined workflow required to tackle the hands-on modernization tasks of the subsequent phases.

Revised Course Architecture

The 20 prescribed sessions are organized into five distinct modules. This architecture is engineered to build progressively from mindset shifts to advanced orchestration, culminating in applied readiness for the laboratory environment.

Module 1: The Agentic Paradigm & CLI Environment (Sessions 1–4)

This module exists to dismantle the unstructured "vibe coding" mindset, establish the role of the engineer as a system orchestrator, and set up the foundational CLI environment. It replaces the legacy "Course Orientation" and generic "LLM Core Concepts" blocks. Model theory is compressed into actionable insights regarding context management, token economics, and predictable degradation. This supports later phases by ensuring every learner possesses a unified, securely configured Claude Code CLI environment and a shared vocabulary around agentic orchestration, preventing technical blockers when entering the labs.

Module 2: Spec-Driven Development & Context Control (Sessions 5–8)

This module teaches the discipline of translating business requirements into constrained, verifiable instructions that AI agents can execute safely. It replaces "Prompting Fundamentals," shifting the focus away from "how to talk to a chatbot" toward "how to write a machine-readable specification" (SDD) and how to manage persistent project rules via CLAUDE.md. This instills the required discipline for the Applied phase, ensuring learners know how to construct Product Requirement Documents (PRDs) that agents can reliably implement without hallucinations or scope creep.

Module 3: Orchestration, MCP & Subagents (Sessions 9–12)

This segment explores how agents interact with external enterprise systems and how engineers can manage multiple agents working in parallel. It completely refactors "Agents & Harnesses" and "Tool Calling & Integration" to focus on the 2026 industry standards: the Model Context Protocol (MCP), deterministic execution hooks, and Claude Code Agent Teams. This supports later phases by preparing learners to connect their AI tools to enterprise databases, internal APIs, and Git repositories securely during hands-on modernization labs.

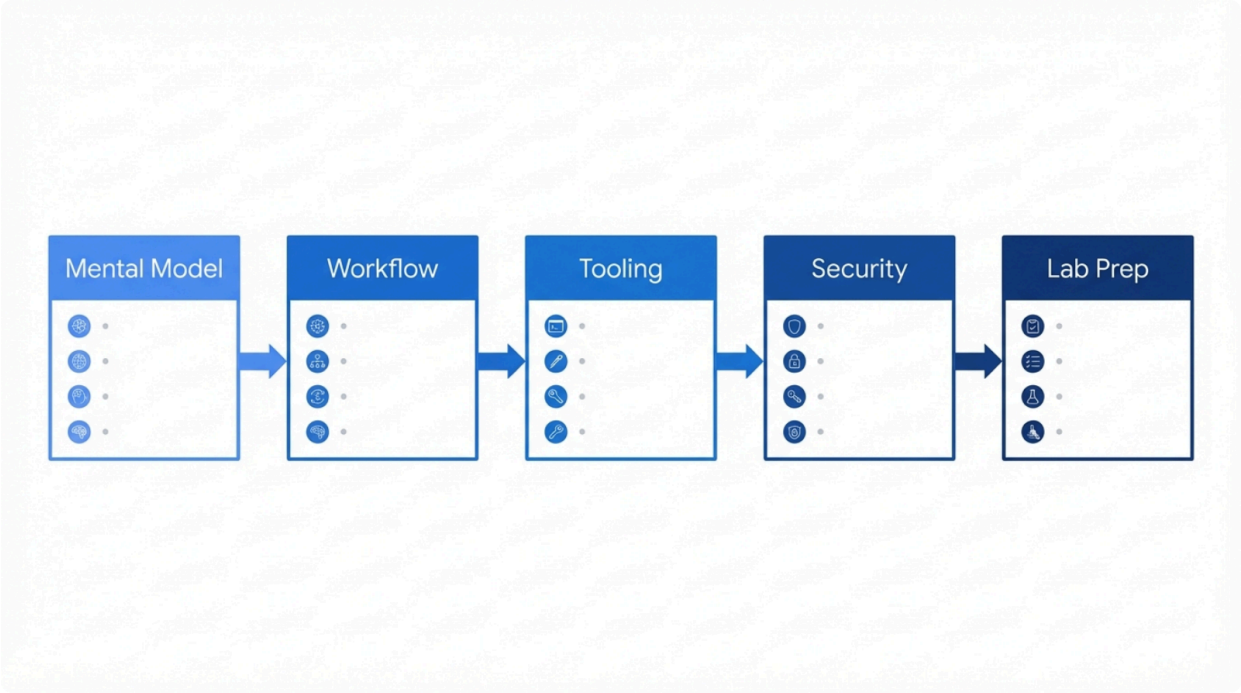
Module 4: Enterprise Security, Governance & Auditability (Sessions 13–16)

This module is designed to secure the execution layer, manage identity for autonomous agents, and ensure all AI-generated code is reviewable and auditable. It upgrades the legacy "Security & Safety" section from basic prompt injection to enterprise-grade agent security, focusing

heavily on Zero Trust, Role-Based Access Control (RBAC), and mitigating Comprehension Debt. This ensures that all code produced in the later Applied phase meets stringent enterprise compliance and security policies.

Module 5: Applied Workflows & Lab Readiness (Sessions 17–20)

The final module simulates real-world software engineering tasks using the fully configured CLI and structured methodologies, acting as the final operational gate before the Labs phase. It replaces the generic "Integrated Capstone" with targeted codebase reconnaissance, test generation, and CI/CD loop integration. This directly sets up the API foundations and codebase analysis tasks that begin on Day 1 of the Labs phase, guaranteeing a seamless transition.



The Foundations phase is architected to rapidly transition engineers from the theoretical agentic paradigm to secure, scalable execution in preparation for Applied Labs.

Full 20-Session Syllabus

The following curriculum table outlines the precise sequence, learning objectives, and operational mechanics for each 23-minute session in the Foundations phase.

Session	Title	1-Sentence Purpose	Learning Objectives	Enterprise Relevance	23-Minute Breakdown	Claude Code Demo	Mini Exercise	Core Resource	Focus Area
1	The Agentic Engineering Shift	Contrast chaotic "vibe coding" with disciplined AI orchestration.	1. Define Agentic Engineering. 2. Identify comprehension debt. 3. Shift from operator to orchestrator.	Untamed AI generation creates unmaintainable, brittle codebases.	0-5: Context 5-15: Comprehension Debt 15-20: Persona shift 20-23: Reflection	*None; foundational mindset setting.*	Identify technical debt vs. comprehension debt in a scenario.	*Beyond Vibe Coding* (Osmani)	Mental Model
2	LLM Internals for Orchestrators	Teach the minimum viable LLM theory needed for engineering decisio	1. Understand tokenization impacts. 2. Grasp probabilistic variance. 3. Define	Engineers must predict when an AI will hallucinate or drop context.	0-5: Transformers brief 5-15: Tokens & Variance 15-20: Contex	Show context compaction triggering in a heavy CLI session.	Calculate token cost for a multi-file refactor.	Anthropic Context Engineering	Mental Model

		ns.	context limits.		t Limits 20-23: Q&A				
3	Context Boundaries & Rot	Explain how agents lose focus and how to manage the integration problem.	1. Identify context rot. 2. Design atomic boundaries. 3. Define the N-tool problem.	Overloading an agent guarantees failure; scoping is critical.	0-5: Context Rot 5-15: NxM Problem 15-20: Scoping 20-23: Review	Provide 50 irrelevant files to Claude and watch reasoning degrade.	Scope a feature request to exactly 3 necessary files.	*AI-Augmented BPM* (Dumas)	Mental Model
4	CLI Architecture & Setup	Install, authenticate, and configure the Claude Code CLI.	1. Execute native installation. 2. Authenticate securely. 3. Navigate CLI architecture.	The CLI is the primary environment for agentic orchestration.	0-5: Architecture 5-15: Install & Auth 15-20: Commands 20-23: Verification	Native install, Auth, /help, /model switching.	Run /doctor and execute a basic read-only query.	Claude Code Quickstart	Tooling

5	Spec-Driven Development	Enforce writing structured specifications before AI code generation.	1. Define SDD methodology. 2. Write testable specs. 3. Use AI to refine requirements.	Code is cheap; intent is expensive. Specs prevent architectural drift.	0-5: SDD Philosophy 5-15: Writing the Spec 15-20: Spec drafting 20-23: Review	Ask Claude to critique a poorly written spec markdown file.	Draft a 10-line PRD for a simple API endpoint.	Thoughtworks: SDD in 2025	Workflow
6	Plan Mode vs. Act Mode	Master the boundary between codebase exploration and execution.	1. Toggle Plan Mode. 2. Execute read-only recon. 3. Approve plans before Act Mode.	Prevents rogue agents from rewriting architectures without oversight.	0-5: Plan vs Act 5-15: Read-only tools 15-20: Plan approval 20-23: Safety Check	claude --permission-mode plan to map a directory.	Generate a plan, critique it, reject it, and refine.	DevTools Academy: Practical Guide	Tooling
7	Context Control via	Establish persistent,	1. Create CLAUDE.md.	Standardizes AI behavi	0-5: Config Hierarchy	Show Claude adhering to a	Write 3 core rules for a	Guide to Claude Code	Workflow

	CLAUDE.md	repo-level governance using configuration files.	2. Enforce constraints. 3. Manage project memory.	or across a team, enforcing DRY and SOLID.	5-15: Writing rules 15-20: Config testing 20-23: Review	strict formatting rule defined in CLAUDE.md.	CLAUDE.md file.	V2	
8	Extending Capabilities with MCP	Connect AI to enterprise data safely using the Model Context Protocol.	1. Define MCP architecture. 2. Understand Servers vs Clients. 3. Connect a basic server.	MCP is the standard for connecting AI to databases and APIs securely.	0-5: What is MCP? 5-15: Client/Server setup 15-20: Tool discovery 20-23: Q&A	Connect a GitHub MCP server to query live enterprise data.	Inspect an MCP JSON-RPC payload.	A Deep Dive Into MCP (a16z)	Tooling
9	Skills & Deterministic Hooks	Automate repetitive quality checks using Skills and	1. Differentiate Skills vs Hooks. 2. Install SKILL.md. 3.	Hooks enforce rules where LLMs fail; Skills scale team experti	0-5: Skills vs Hooks 5-15: Skill installation	Use the frontend-design skill to force architectural consist	Write a basic Hook to run a linter before AI commits.	10 Must-Have Skills for Claude	Tooling

		Hooks.	Configure pre-commit Hooks.	se.	15-20: Hook config 20-23: Review	ency.			
10	Task Decomposition & Subagents	Break massive workloads into atomic tasks utilizing isolated subagent contexts.	1. Define atomic decomposition. 2. Spawn single-session subagents. 3. Isolate context.	Complex tasks fail in single prompts; decomposition guarantees reliability.	0-5: Atomic Tasks 5-15: Spawning Subagents 15-20: Task routing 20-23: Reflection	Spawn a security-reviewer subagent to analyze a single file.	Decompose a feature request into 3 isolated tasks.	Paul Duvall: AI Dev Patterns	Workflow
11	Parallel Orchestration: Agent Teams	Coordinate multiple independent Claude sessions using Agent Teams.	1. Configure Team Leads. 2. Manage the Task List. 3. Oversee peer-to-peer messa	Speeds up SDLC by running frontend, backend, and QA in parallel.	0-5: Teams vs Subagents 5-15: Shared Mailbox 15-20: Orchestration	Spin up 3 teammates (UI, API, QA) to simultaneously build a feature.	Design a 3-agent team architecture for a specific ticket.	Claude Code Agent Teams Docs	Tooling

			ging.		20-23: Q&A				
12	OWASP Top 10 for Agentic Apps	Mitigate the primary execution-layer threats in AI systems.	1. Shift to mal-action focus. 2. Identify indirect prompt injection. 3. Analyze tool misuse.	Agents interact with real infrastructure; flaws here cause breaches.	0-5: Agentic Threats 5-15: OWASP Top 10 15-20: Threat modeling 20-23: Review	*None; architectural threat modeling focus.*	Identify the OWASP vulnerability in a provided agent trace.	OWASP Agentic Top 10	Security
13	Zero Trust & Permission Boundaries	Implement strict Least Privilege access for AI agents in the CLI.	1. Configure tool whitelists. 2. Apply RBAC to AI identities. 3. Manage approval modes.	Prevents blast-radius damage if an agent hallucinates or is injected.	0-5: Zero Trust for AI 5-15: CLI Permissions 15-20: Boundary setting 20-23: Reflect	Show a blocked execution due to a strict --permission-mode setting.	Write a strict tool whitelist for a code review agent.	Heights CG: AI Security	Security

					ion				
14	Secure MCP Governance	Enforce data privacy when connecting agents to enterprise backends.	1. Secure MCP endpoints. 2. Filter data provenance. 3. Prevent data exfiltration.	Connecting AI to databases requires strict data layer enforcement.	0-5: MCP Risks 5-15: Gateway patterns 15-20: Risk Assessment 20-23: Q&A	Connect a simulated "HR Database" MCP with read-only scoped access.	Conduct a risk review of a proposed Jira MCP integration.	State of AI Agent Security	Security
15	Code Review & Cognitive Debt Mitigation	Maintain deep system understanding over AI-generated code.	1. Perform structural diff reviews. 2. Mitigate automation bias. 3. Enforce the "80/20" rule.	Senior engineers must catch the 20% of subtle bugs AI introduces.	0-5: Automation Bias 5-15: Diff Analysis 15-20: Review exercise 20-23: Reflection	Generate a PR, then manually review the diff for subtle logic flaws.	Find hallucinated logic in a 50-line AI-generated PR.	The Engineering Leader's Truth	Security

16	Auditability & Workflow Narration	Generate immutable records for all agentic actions.	1. Capture agent traces. 2. Enforce PR narration. 3. Align with compliance audits.	Regulatory compliance requires explicit proof of human oversight.	0-5: Compliance Needs 5-15: Logging & Traces 15-20: Narration drafting 20-23: Review	Export a session log and attach it to a PR description.	Write a "PR Narration" explaining an AI's architectural choice.	NIST AI Agent Standards	Security
17	Lab Prep: Codebase Reconnaissance	Utilize read-only AI tools to rapidly comprehend enterprise legacy code.	1. Execute AI Glob/Grep. 2. Generate architecture maps. 3. Trace execution flows.	The first step in any real-world ticket is understanding the system.	0-5: Recon Strategy 5-15: Navigating legacy 15-20: Recon execution 20-23: Next steps	Use Plan Mode to analyze a repo and generate a dependency map.	Find where authentication is handled in a dummy repo.	Claude Code Workflows	Lab Prep
18	Lab Prep:	Command	1. Identif	AI excels	0-5: TDD	Point Claude	Prompt an	Practical AI	Lab Prep

	Test Generation & Coverage	agents to generate robust, edge-case-aware tests.	y coverage gaps. 2. Generate test scaffolds. 3. Execute TDD agent loops.	at edge-case test generation, improving safety and velocity.	with AI 5-15: Generating suites 15-20: Edge case tests 20-23: Review	at an untested function to write and run a suite.	agent to test a specific edge-case constraint.	Dev Patterns	
19	Lab Prep: The CI/CD Agentic Loop	Integrate CLI agents with pipelines for automated triage.	1. Read pipeline logs. 2. Automate bug triage. 3. Manage the /loop command.	AI drastically reduces time-to-resolution for broken builds.	0-5: CI/CD Triage 5-15: Log analysis 15-20: Triaging failures 20-23: Reflection	Pipe a failing build log (`cat log	claude `) and ask for a fix.	Diagnose a CI failure log using Claude Code.	CircleCI: AI SDLC
20	Readiness Assessment & Capstone	Verify safe CLI practices before	1. Consolidate workflows. 2. Execut	Ensures learners meet the "Ready	0-5: Framework review 5-15:	End-to-end: Spec -> Plan -> Approv	Complete the Foundations Readiness	*AISDLC 2026 Maturity*	Lab Prep

	Setup	advancing to Applied Labs.	e a full SDD loop. 3. Transition to Applied Phase.	for Labs" security baseline.	End-to-end demo 15-20: Readiness check 20-23: Next Steps	e -> Act -> Test -> Review.	Checklist.		

Session-Level Blueprint Detail

Module 1: The Agentic Paradigm & CLI Environment

Session 1: The Agentic Engineering Shift

This session introduces the fundamental difference between ad-hoc "vibe coding" and disciplined "agentic engineering." The instruction focuses heavily on the hidden costs of artificial intelligence: technical debt and, more insidiously, comprehension debt. The instructor must adopt the persona of a battle-scarred engineering lead, explicitly stating that vibe coding is suitable only for weekend hackathons, whereas agentic engineering is required for enterprise payroll and production stability. A primary misconception to correct is the belief that because AI writes the code, engineers no longer need to understand system architecture; the instructor will correct this by establishing that AI writes syntax, but human engineers own the architecture and the liability. No CLI demo is utilized here, preserving a pure architectural whiteboard focus.

- **Exercise:** Review a scenario where a team shipped a feature ten times faster but cannot debug a resulting production outage. Identify the specific areas of "comprehension debt."
- **Discussion Prompt:** When is vibe coding temporarily acceptable, and at what specific point does it become a corporate liability?
- **Homework:** Read *Vibe Coding is Not the Same as AI-Assisted Engineering* (Osmani).
- **Survey Question:** On a scale of 1-5, how confident are you in taking personal legal and professional ownership of code written entirely by an AI agent?

Session 2: LLM Internals for Orchestrators

This module strips away deep machine learning theory to focus strictly on what impacts day-to-day engineering: token limits, probabilistic variance, and context window mechanics. The instructor approaches the LLM as a highly capable but highly forgetful junior developer with a strict, measurable short-term memory limit. A critical misconception to address is the

anthropomorphic assumption that models actually *understand* the code they write; the instructor will clarify that models merely predict statistically likely token sequences based on the provided context. The CLI demonstration involves running an artificially heavy data session until Claude triggers a "context compaction" event, demonstrating memory limits and automated summarization in real-time.

- **Exercise:** Calculate the approximate token cost footprint and latency impact of loading a specific, massive framework documentation set into a single prompt.
- **Discussion Prompt:** How does probabilistic variance mathematically impact the reliability of automated CI/CD pipelines?
- **Homework:** Review Anthropic's technical documentation on Context Windows and Token Economics.
- **Survey Question:** Have you ever experienced an AI "forgetting" crucial architectural instructions mid-conversation?

Session 3: Context Boundaries & Rot

This session explores how to strictly scope tasks to prevent "context rot"—the phenomenon where an LLM becomes confused and hallucinates due to excessive, irrelevant data. It also introduces the NxM integration problem inherent in scaling agentic tools. The instructor will emphasize that "more context" is not inherently better; optimizing the signal-to-noise ratio is the key to agent accuracy. The common misconception that engineers should "just give the AI access to the whole repository" will be actively dismantled. The CLI demonstration will feature providing Claude with 50 irrelevant files and asking a targeted architectural question, visibly demonstrating how reasoning degrades compared to a cleanly scoped, 3-file context window.

- **Exercise:** Given a complex feature ticket, list the exact 3-4 files required for context, explicitly justifying why the rest of the repository must be excluded.
- **Discussion Prompt:** Why do distributed, multi-agent systems mathematically perform better than single, monolithic mega-agents?
- **Homework:** Read *AI-Augmented BPM* (Dumas) regarding system boundary definitions.
- **Survey Question:** What is your current, standardized strategy for deciding which code snippets to share with an AI assistant?

Session 4: Claude Code CLI Architecture & Setup

This is a strictly hands-on session focusing on the installation, authentication, and configuration of the Claude Code CLI within enterprise environments. The instructor will champion the CLI as the professional's native environment, arguing that moving from a web user interface to the terminal is the mandatory first step toward serious agentic orchestration. The misconception that developers can rely solely on integrated development environment (IDE) chat extensions will be corrected; IDE chats are constrained, whereas the CLI enables shell scripting, piping, and headless continuous integration. The demonstration will walk through a native installation, secure OAuth authentication, and fundamental commands like /help and /model switching.

- **Exercise:** Authenticate the CLI in an isolated dummy repository and run a basic,

read-only /doctor diagnostic command.
- **Discussion Prompt:** What are the specific enterprise security implications of executing terminal-based AI versus browser-based AI?
- **Homework:** Complete the official Claude Code Quickstart guide and verify local installation.
- **Survey Question:** How comfortable are you executing and debugging complex operational workflows purely from the command line?

Module 2: Spec-Driven Development & Context Control

Session 5: Spec-Driven Development (SDD)

This session formally introduces the Spec-Driven Development methodology, teaching engineers how to write machine-readable, highly testable specifications before permitting any code generation. The instructor's angle is definitive: the AI is merely the typist; the Specification is the immutable blueprint. If the generated output is flawed, the specification was flawed. The misconception that "writing specs slows down AI development" will be countered with the reality that specs are the only mechanism to prevent infinite, costly AI rework loops. The CLI demonstration involves feeding Claude a vague, unstructured prompt versus a highly structured markdown Specification, visibly comparing the execution accuracy and edge-case handling of both outputs.

- **Exercise:** Draft a concise, 10-line markdown Product Requirement Document (PRD) for a user authentication endpoint, containing strict, binary acceptance criteria.
- **Discussion Prompt:** How does Spec-Driven Development fundamentally change the daily role of a traditional software engineer?
- **Homework:** Read Thoughtworks' piece on the necessity of Spec-Driven Development in 2025/2026.
- **Survey Question:** How often do you currently write a formal, version-controlled specification before prompting an AI to write code?

Session 6: Plan Mode vs. Act Mode

This session provides a deep dive into Claude Code's dual operational modes. It teaches the absolute discipline of using read-only exploration (Plan Mode) to generate and refine architectural proposals before ever granting write access (Act Mode). The instructor will frame Plan Mode as the ultimate enterprise safety net, forcing the AI to "show its math" and intended file modifications before it touches the codebase. The dangerous misconception of "I'll just let it code and fix the bugs later" will be actively discouraged. The CLI demonstration will utilize claude --permission-mode plan to map a directory and propose a complex refactor without actually editing any files, showcasing the approval workflow.

- **Exercise:** Generate a refactoring plan using the CLI. Actively critique the plan, force Claude to refine it based on edge cases, and only then approve the transition to Act Mode.

- **Discussion Prompt:** Why is mathematically separating "thinking" (planning) from "typing" (execution) crucial in autonomous enterprise systems?
- **Homework:** Practice toggling Plan Mode on a complex personal or open-source repository.
- **Survey Question:** Have you ever had an AI overwrite a critical file incorrectly because you lacked the ability to review its execution plan first?

Session 7: Context Control via CLAUDE.md

This session explores the configuration hierarchy of agentic tools, focusing on writing project-level rules in CLAUDE.md to enforce styling, architecture, and constraints autonomously. The instructor will position the CLAUDE.md file as an automated Tech Lead that persistently enforces the team's "definition of done" without requiring constant human intervention or repetitive prompting. The misconception that developers must paste a massive system prompt into every new chat window will be corrected. The CLI demonstration will show Claude strictly adhering to a hyper-specific formatting rule defined exclusively in CLAUDE.md (e.g., "Always use single quotes and trailing commas in JSON payloads").

- **Exercise:** Draft three core, immutable architectural rules for a CLAUDE.md file designed for a strict, highly regulated enterprise microservice.
- **Discussion Prompt:** What specific types of instructions belong in a global ~/.claude/CLAUDE.md versus a project-local configuration file?
- **Homework:** Create a functional CLAUDE.md for a current, active work project and observe the behavioral shift in the agent.
- **Survey Question:** Does your engineering team currently utilize a standardized, version-controlled method to share AI prompting rules and constraints?

Session 8: Extending Capabilities with MCP

This session introduces the Model Context Protocol (MCP), framing it as the "USB-C for AI." It explains how MCP securely connects autonomous agents to enterprise databases, internal tools, and proprietary APIs without custom, brittle integration code. The instructor will highlight that MCP is the mechanism that elevates a smart autocomplete tool into a fully integrated enterprise agent, solving the pervasive fragmentation problem. The misconception that teams must write custom API wrappers for every single AI tool will be definitively corrected. The CLI demonstration will feature connecting a pre-built MCP server (such as GitHub or a local Postgres instance) and asking Claude Code to autonomously query a live database row or issue ticket.

- **Exercise:** Review the raw JSON-RPC payload of an MCP tool call to understand exactly how context is structured and where authorization headers are securely applied.
- **Discussion Prompt:** Why is an open standard like MCP critical for enterprise AI adoption compared to relying on vendor-locked plugin ecosystems?
- **Homework:** Read the a16z analysis, *A Deep Dive Into MCP*.
- **Survey Question:** Prior to this course, how familiar were you with the Model Context

Protocol (MCP) and its role in enterprise architecture?

Module 3: Tooling, MCP, and Agentic Orchestration

Session 9: Skills & Deterministic Hooks

This session distinguishes between probabilistic workflows (defined in SKILL.md) and deterministic, pre-commit Hooks. It teaches engineers how to enforce rigid quality gates automatically, ensuring AI code cannot be committed if it fails basic enterprise standards. The instructor will emphasize that engineers should never ask an LLM to "remember" to run a linter; instead, they must use a Hook to force the linter to run algorithmically every time. The misconception that AI is smart enough to consistently check its own work reliably will be dismantled. The CLI demonstration will involve installing a frontend-design skill, followed by configuring a Hook that automatically runs a test suite, blocking Claude from committing if the tests fail.

- **Exercise:** Write a basic JSON configuration for a Hook that intercepts the commit process and blocks execution if a plaintext secret or API key is detected in the diff.
- **Discussion Prompt:** Where is the exact boundary between where AI flexibility (Skills) is advantageous, and where rigid engineering rules (Hooks) must take over?
- **Homework:** Install a verified third-party skill from the Anthropic repository and execute it locally.
- **Survey Question:** Do you currently utilize any automated CI/CD checks specifically designed to validate or gate AI-generated code before it enters the main branch?

Session 10: Task Decomposition & Subagents

This session teaches the critical skill of atomic decomposition. It demonstrates how to spawn isolated, single-session subagents to tackle specific, narrow tasks without polluting the main context window or confusing the primary agent. The instructor will hammer home the reality that massive, monolithic prompts fail; complex work must be broken down and delegated to "specialist" subagents. The misconception that a single prompt can reliably build a full-stack application will be corrected. The CLI demonstration will show the spawning of a security-reviewer subagent to analyze a single file, highlighting how it returns only a concise summary to the main session, preserving token limits.

- **Exercise:** Take a highly complex feature request (e.g., "Implement an OAuth2 billing dashboard") and decompose it into three strictly isolated subagent tasks with defined inputs and outputs.
- **Discussion Prompt:** How does strict subagent isolation mathematically reduce token costs while simultaneously improving the deterministic accuracy of the overall system?
- **Homework:** Read Paul Duvall's *AI Development Patterns* focusing on atomic decomposition.
- **Survey Question:** What is the largest, most complex coding task you have successfully delegated to an AI in a single, unbroken prompt?

Session 11: Parallel Orchestration: Agent Teams

This session explores the bleeding edge of orchestration: Opus 4.6 Agent Teams. It teaches how to coordinate multiple Claude sessions working simultaneously, utilizing shared task lists and peer-to-peer messaging. The instructor will emphasize that the developer has evolved from managing a single junior freelancer to orchestrating a synchronized, multi-disciplinary development crew. The misconception that Subagents and Agent Teams are identical will be corrected; Teams run in parallel, communicate directly peer-to-peer, and share state, whereas subagents are strictly hierarchical. The CLI demonstration will spin up a 3-agent team (UI, API, QA) to simultaneously research and draft a feature based on a single initial command.

- **Exercise:** Design a 3-agent team architecture on paper, explicitly defining the specific roles, context limits, and mailbox communication flow required to resolve a complex Jira ticket.
- **Discussion Prompt:** What are the inherent systemic risks of autonomous agents communicating directly with one another without human intervention or oversight?
- **Homework:** Review the official Claude Code Agent Teams documentation regarding mailbox management.
- **Survey Question:** How comfortable are you with the concept of multiple AI agents interacting with each other, generating tokens and executing logic in the background?

Session 12: OWASP Top 10 for Agentic Apps

This session pivots sharply to security, examining the OWASP Top 10 for Agentic Applications. The focus is on the fundamental shift in threat modeling: moving from "mal-content" (the AI generating bad text) to "mal-action" (the AI autonomously exfiltrating data or abusing tools). The instructor will stress that while a hallucination is merely annoying, an agent autonomously dropping a production database table due to an indirect prompt injection is a career-ending breach. The misconception that basic prompt filtering is sufficient to secure AI will be aggressively dismantled. There is no CLI demo in this session; it is purely an architectural threat modeling exercise utilizing real-world 2026 red-team scenarios.

- **Exercise:** Analyze a provided log trace of an agent executing an unauthorized API call and identify the specific OWASP vulnerability (e.g., Indirect Prompt Injection, Insecure Output Handling) that enabled the breach.
- **Discussion Prompt:** How does agentic autonomy fundamentally alter the traditional enterprise threat surface compared to legacy static applications?
- **Homework:** Read the full *OWASP Top 10 for Agentic Applications (2026)* documentation.
- **Survey Question:** Does your organization currently have specific, documented security policies governing the deployment of autonomous AI agents?

Module 4: Enterprise Security, Governance & Auditability

Session 13: Zero Trust & Permission Boundaries

This session applies Zero Trust principles directly to the CLI environment. It teaches engineers

how to configure strict tool whitelists, apply Role-Based Access Control (RBAC) to AI identities, and manage interactive approval modes. The instructor will mandate treating the agent like an untrusted, highly privileged third-party contractor: it must be given exactly the permissions it needs, and nothing more. The dangerous misconception that running the CLI with --dangerously-skip-permissions is an acceptable way to save time will be harshly corrected. The CLI demonstration will showcase a blocked execution, where Claude attempts an operation outside its --permission-mode bounds and is categorically denied by the system.

- **Exercise:** Write a strict, JSON-formatted tool whitelist configuration for a read-only code review agent that explicitly denies Bash execution and file system writes.
- **Discussion Prompt:** How do engineering teams balance the desire for maximum developer velocity with the absolute necessity of strict agent permissioning?
- **Homework:** Review Heights CG's *AI Security Best Practices* focusing on the execution layer.
- **Survey Question:** Have you ever run an AI coding tool or agent with global or administrator privileges on your local development machine?

Session 14: Secure MCP Governance

This session focuses entirely on the data layer, teaching how to secure MCP endpoints to prevent data poisoning and exfiltration when agents connect to sensitive enterprise backend systems. The instructor will highlight that while MCP is incredibly powerful, an unsecured MCP server acts as an open backdoor straight into the enterprise's most critical data. The misconception that "if the AI is hosted internally, the data connection is inherently safe" will be corrected. The CLI demonstration will connect a simulated "HR Database" MCP server configured with strict, read-only scoped access to demonstrate data layer enforcement in action.

- **Exercise:** Conduct a rapid security risk review of a proposed Jira MCP integration, identifying potential privilege escalation vectors if an agent is compromised via indirect prompt injection from a malicious ticket description.
- **Discussion Prompt:** Why must absolute security constraints be enforced at the MCP server level rather than relying on LLM prompt-level filtering?
- **Homework:** Read *The State of AI Agent Security 2026* report.
- **Survey Question:** Who holds the ultimate responsibility for securing AI integrations and data access patterns in your organization?

Session 15: Code Review & Cognitive Debt Mitigation

This session addresses the human element of AI security, focusing on structural diff reviews, combating automation bias, and the critical necessity of catching the subtle 20% of logic bugs that AI introduces. The instructor will clarify that the modern engineer's job is no longer just to write code; it is to mathematically prove the code works. Blindly approving AI PRs is framed as negligence. The misconception that "if the AI wrote it, and the tests passed, it must be structurally sound" will be corrected. The CLI demonstration will use Claude to generate a PR,

followed by a manual review of the diff to find a subtle, hallucinated logic flaw that successfully passed basic unit testing.

- **Exercise:** Analyze a 50-line AI-generated PR. Identify the architectural violation or business-logic flaw hidden within the otherwise perfectly formatted syntax, and draft the required review comment.
- **Discussion Prompt:** How do engineering teams maintain deep system understanding and avoid cognitive debt when AI is authoring 80% or more of the codebase?
- **Homework:** Read *The Engineering Leader's Truth about AI* regarding automation bias.
- **Survey Question:** Have you ever approved an AI-generated code snippet or PR without fully understanding the underlying mechanics of how it functioned?

Session 16: Auditability & Workflow Narration

This session focuses on compliance and traceability, teaching how to generate immutable records of agent actions and enforcing human narration of AI architectural choices. The instructor will emphasize that if an auditor or compliance officer asks why a system made a specific decision, "the AI did it" is an legally unacceptable answer. The engineer must provide the trace. The misconception that AI development doesn't require a paper trail if the final product works will be corrected. The CLI demonstration will export a complete Claude Code session log and attach it to a PR description to demonstrate workflow traceability and compliance.

- **Exercise:** Write an auditable "PR Narration" summarizing exactly *why* an AI agent chose a specific database index pattern, ensuring the justification meets strict enterprise compliance standards.
- **Discussion Prompt:** How do emerging global regulations (such as the EU AI Act) directly impact how we must document and audit AI-assisted engineering workflows?
- **Homework:** Review the *NIST AI Agent Standards Initiative* regarding traceability.
- **Survey Question:** Does your engineering team currently utilize tooling to track exactly which lines of code in your repository were generated by AI versus humans?

Module 5: Applied Workflows & Lab Readiness

Session 17: Lab Prep: Codebase Reconnaissance

This session transitions the curriculum from theory to practical application. It uses read-only Plan Mode and CLI tools (like Glob and Grep) to rapidly map and comprehend unfamiliar, messy enterprise legacy codebases. The instructor will frame the AI as the ultimate codebase archaeology tool, emphasizing that the first step in any real-world ticket is understanding the legacy monster before attempting to modify it. The misconception that developers should manually read the entire source code to learn a system will be corrected. The CLI demonstration will use Plan Mode to analyze a 100-file repository, successfully tracing the execution flow of a specific, undocumented transaction.

- **Exercise:** Use the CLI in Plan Mode to rapidly find exactly where user authentication and

token validation are handled in a provided, highly fragmented dummy repository.
- **Discussion Prompt:** How does AI-assisted codebase reconnaissance fundamentally alter the timeline and effectiveness of the developer onboarding process?
- **Homework:** Use Claude Code to map the core architecture of a small, unfamiliar open-source project.
- **Survey Question:** Historically, how long does it typically take you to feel fully productive and confident in a completely new, undocumented enterprise codebase?

Session 18: Lab Prep: Test Generation & Coverage

This session focuses on utilizing agents for Test-Driven Development (TDD). It instructs agents to identify coverage gaps, generate complex test scaffolds, and execute tests autonomously. The instructor will dictate the workflow: let the AI write the tests, let the AI run the tests, but the human engineer must review and own the coverage strategy. The misconception that AI is only useful for writing feature code will be corrected; AI excels at tedious edge-case test generation. The CLI demonstration will point Claude at a complex, untested function and command it to write a comprehensive test suite, run it, and fix any resulting failures autonomously.

- **Exercise:** Prompt an agent to specifically write tests that target a difficult boundary constraint or mathematical edge-case in a provided code snippet.
- **Discussion Prompt:** Does AI-generated testing lower the perceived value of QA engineers, or does it elevate their role from script writers to high-level test strategists?
- **Homework:** Have Claude Code generate comprehensive unit tests for an untested file in a personal project.
- **Survey Question:** How confident are you currently in relying on an AI to generate robust, edge-case-aware unit tests for mission-critical code?

Session 19: Lab Prep: The CI/CD Agentic Loop

This session integrates CLI agents directly with continuous deployment pipelines. It teaches engineers how to pipe failing build logs directly to Claude and automate bug triage using the /loop command. The instructor will demand that engineers stop parsing 10,000-line Jenkins logs manually; they must pipe it to the agent and ask for the specific fix. The misconception that AI is strictly a pre-commit, local development tool will be corrected. The CLI demonstration will feature piping a massive, failing build log (cat build_log.txt | claude) and asking Claude to diagnose the error and implement a targeted, single-file fix.

- **Exercise:** Given a raw, complex CI failure log, use Claude Code to instantly identify the root cause of the failure and propose the exact terminal command required to fix it.
- **Discussion Prompt:** How does automated, agentic triage impact Mean Time To Recovery (MTTR) metrics during critical production incidents?
- **Homework:** Read CircleCI's *AI SDLC* overview regarding the collapse of traditional SDLC phases.
- **Survey Question:** Have you ever successfully used an LLM or agent to debug a failing,

complex CI/CD pipeline issue?

Session 20: Readiness Assessment & Capstone Setup

The final session acts as the definitive gate. It consolidates the structured workflows, verifies safe CLI practices, and officially transitions learners to the Applied Labs phase. The instructor will emphasize that the learners now possess the tools, the rules, and the safety net required to build real enterprise software. The misconception that "I've finished Foundations, I can go back to vibe coding now" will be sternly corrected; Foundations established the constraints, and the subsequent Labs will ruthlessly enforce them. The CLI demonstration will feature a rapid, end-to-end workflow: Write a Spec -> Enter Plan Mode -> Approve -> Enter Act Mode -> Generate Tests -> Review Diff.

- **Exercise:** Complete a rigorous self-assessment verifying secure CLI configuration, accurate CLAUDE.md setup, and total comprehension of Zero Trust security principles.
- **Discussion Prompt:** Final Q&A clarifying all expectations, grading rubrics, and operational protocols for the upcoming Applied coaching phases.
- **Homework:** Prepare all local environments, API keys, and repository accesses for Day 1 of the Labs phase.
- **Survey Question:** Do you feel technically equipped and organizationally authorized to use Claude Code safely within a highly regulated enterprise codebase?

What to Keep / Compress / Remove / Add

Keep

- **Prompt Injection & Security Focus:** The emphasis on securing AI systems is vital and must be retained. However, it must be significantly upgraded to cover execution-layer threats (e.g., unauthorized API calls) rather than just conversational prompt injection.
- **Chatbot vs. Raw Model Call:** This conceptual distinction remains an excellent pedagogical anchor for explaining why deterministic CLI tools and API integrations behave fundamentally differently than conversational web interfaces.
- **Capstone PR Progression:** Moving the learner toward a tangible, reviewable artifact (the Pull Request) is the correct outcome and must remain the structural goal of the course.

Compress

- **Transformer Mental Model:** Enterprise engineers do not need to understand attention heads, vector mathematics, or self-attention mechanisms. Compress this heavily into a 5-minute functional overview focused solely on "how models process and drop context."
- **Training vs. Inference:** Condense this into a brief conceptual note. Knowing how a model was pre-trained does not materially change how an engineer orchestrates an agent in a 2026 CLI environment.
- **Manual Prompting Fundamentals:** Aggressively compress time spent teaching developers "how to ask nicely." Modern CLI tools utilize system prompts, CLAUDE.md

hierarchies, and MCP servers; manual prompt wrangling is rapidly becoming a legacy skill.

Remove

- **"Vibe Coding" Acceptance:** Remove any implied tolerance for unstructured, prompt-and-pray workflows. The course must take a hard, opinionated stance against it as a liability in enterprise environments.
- **Generic AI Ethics:** Remove abstract ethical philosophy discussions. Replace them entirely with concrete engineering constraints regarding data privacy, data exfiltration, and compliance frameworks (e.g., EU AI Act, SOC2 readiness for AI).
- **"Agents vs Workflows" as a Binary:** In 2026, agents *are* the workflow. Remove the outdated distinction and focus instead on the orchestration of multi-agent teams.

Add

- **Claude Code CLI as Core Platform:** Every single session must interface with or relate directly to the Claude Code CLI, establishing it as the primary engineering sandbox.
- **Spec-Driven Development (SDD):** Add formal, rigorous instruction on writing markdown specifications *before* any code generation is permitted.
- **Model Context Protocol (MCP):** Add deep, technical dives into MCP architecture, JSON-RPC payloads, server configuration, and secure enterprise integration.
- **Plan Mode vs. Act Mode Workflow:** Explicitly teach the UI/UX boundaries and security implications of read-only exploration versus write-enabled execution.
- **Opus 4.6 Agent Teams:** Add advanced curriculum on parallel orchestration, subagent spawning, and managing shared mailbox communication patterns between autonomous agents.

Survey Design

Pre-Course Survey (12 Questions)

Purpose: To baseline the cohort's workflow maturity, perceived risk, and current friction points before instruction begins.

1. **Confidence:** On a scale of 1-5, how confident are you in relying on AI to write production-ready code without heavy manual refactoring?
2. **Current Usage:** How many hours per week do you currently use AI coding assistants? (0, 1-5, 5-10, 10+)
3. **Current Usage:** Do you primarily use AI via a Chatbot Web UI, an IDE integration (e.g., Copilot), or a headless CLI (e.g., Claude Code)?
4. **Workflow Maturity:** Do you currently write formal specifications (e.g., PRDs, Architecture docs) *before* prompting an AI to generate code? (Always, Sometimes, Rarely, Never)
5. **Workflow Maturity:** Are you familiar with the Model Context Protocol (MCP) and its role in enterprise architecture? (Yes/No)

6. **Friction:** What is the biggest frustration you face when using AI for coding? (Hallucinations, Context limits, Setup/Integration, Security blocks)
7. **Friction:** How often do you find yourself spending more time reviewing and fixing AI-generated code than it would have taken to write it yourself?
8. **Perceived Risk:** On a scale of 1-5, how concerned are you about introducing subtle security vulnerabilities via AI-generated code?
9. **Perceived Risk:** Does your organization have a formal, documented policy regarding the use of autonomous AI agents accessing internal repositories?
10. **Manager Blocker:** Does your leadership actively mandate AI usage for productivity, or restrict its use due to security and compliance concerns?
11. **Value:** What is the primary business value you hope to gain from this course? (Velocity, Quality, Security, Legacy Modernization)
12. **Demographic:** What is your primary engineering role? (Frontend, Backend, Platform, Tech Lead, Engineering Manager)

Mid-Course Survey (9 Questions)

Purpose: To check comprehension of core concepts, monitor CLI adoption rates, and verify the shift in mental models.

1. **Confidence:** Are you comfortable navigating, authenticating, and configuring the Claude Code CLI? (1-5)
2. **Workflow Maturity:** Have you successfully executed a strictly separated "Plan Mode" to "Act Mode" workflow during the exercises? (Yes/No)
3. **Workflow Maturity:** Do you clearly understand the operational and liability differences between "vibe coding" and "agentic engineering"? (Yes/No)
4. **Friction:** Which technical concept has been the most difficult to grasp so far? (Context Rot, SDD, MCP Architecture, CLAUDE.md Hierarchy, Subagents)
5. **Perceived Risk:** After reviewing the OWASP Top 10 for Agentic Apps, is your perception of enterprise AI security risk higher, lower, or exactly the same?
6. **Value:** Have you identified an immediate, practical use case for a CLAUDE.md configuration in your daily production work?
7. **Pacing:** Is the technical depth of the CLI demonstrations too basic, perfectly balanced, or too advanced?
8. **Application:** Have you successfully installed and executed any Claude Code Skills (e.g., frontend-design) locally?
9. **Feedback:** What is one specific topic or workflow the instructor should spend more time detailing in the remaining sessions?

Post-Course Survey (12 Questions)

Purpose: To measure ROI, verify technical readiness for the Applied phase, and identify lingering organizational blockers.

1. **Confidence:** On a scale of 1-5, how confident are you in deploying structured, secure

agentic workflows within your enterprise environment?
2. **Confidence:** Do you feel technically equipped and ready to enter the hands-on Labs phase? (Yes/No)
3. **Workflow Maturity:** Will you formally adopt Spec-Driven Development (SDD) as a mandatory step before allowing AI code generation? (Yes/No)
4. **Current Usage:** Do you plan to transition your primary AI workflow away from a Web/IDE interface toward a CLI agent?
5. **Friction:** What do you anticipate will be the biggest cultural or technical challenge in implementing Agentic Engineering at your company?
6. **Perceived Risk:** Do you feel equipped to rigorously review AI-generated pull requests and mitigate "comprehension debt" across your team? (1-5)
7. **Manager Blocker:** Will you need to request infrastructure changes (e.g., firewall rules, specific IAM roles) to implement MCP servers at work?
8. **Observed Value:** Did this course successfully differentiate rigorous, enterprise-safe AI engineering from generic, hype-driven beginner AI courses?
9. **Observed Value:** What was the single most valuable module, session, or concept introduced in the Foundations phase?
10. **Application:** Are you comfortable spinning up an Opus 4.6 Agent Team for parallel task execution and managing their shared mailbox?
11. **Application:** Can you confidently explain to a peer or auditor why Zero Trust principles must apply to autonomous AI agents?
12. **Feedback:** Would you recommend this course to a fellow senior engineer, platform architect, or tech lead?

Exercises and Applied Artifacts

The following 12 exercises are distributed across the 20 sessions, designed specifically for the Foundations phase. They focus intensely on reasoning, configuration, and constraint enforcement rather than full-scale coding (which is reserved for the Labs).

1. **The Comprehension Debt Audit (Session 1):** Analyze a provided incident report where an AI-generated database migration caused a severe production outage. Identify exactly where human review failed and document the specific "comprehension debt" that led to the incident.
2. **Context Scoping Blueprint (Session 3):** Given a massive, monolithic repository structure, select the exact four file paths and two documentation URLs needed to give an agent sufficient context to build a specific API route, intentionally and explicitly omitting the rest to prevent context rot.
3. **The 10-Line PRD (Session 5):** Practice Spec-Driven Development by writing a rigid, highly testable Product Requirement Document (PRD) in markdown for a simple feature, ensuring zero ambiguity for an AI agent.
4. **Plan Mode Interrogation (Session 6):** Use Claude Code's Plan Mode to analyze a provided, complex Python script. Force the AI to output a refactoring plan, actively

critique it, and reject it twice based on edge cases before finally approving execution.

5. **CLAUDE.md Drafting (Session 7):** Write a strict CLAUDE.md file that enforces a specific architectural pattern (e.g., Hexagonal Architecture) and mandates standardized logging practices across all agentic outputs.
6. **MCP Payload Inspection (Session 8):** Review a raw JSON-RPC payload generated from a GitHub MCP server. Identify exactly how the context is structured and where the authorization headers are securely applied.
7. **Hook Configuration (Session 9):** Write a JSON configuration for a pre-commit Hook that triggers a gitleaks scan, blocking the AI from executing a commit if any secrets are detected.
8. **Atomic Task Decomposition (Session 10):** Break down a massive, complex Epic ("Implement OAuth2 Billing Dashboard") into four distinct, isolated tasks suitable for delegation to single-session subagents.
9. **Agent Team Architecture (Session 11):** Design a detailed workflow diagram detailing exactly how three Agent Teammates (UI, Backend, QA) will coordinate via a shared mailbox to complete a specific, complex user story.
10. **Zero Trust Whitelisting (Session 13):** Write a strict tool whitelist configuration for a Claude Code session that permits only read access and explicitly disables all Bash execution and file modifications.
11. **The 80/20 Diff Review (Session 15):** Review a provided AI-generated Pull Request where the syntax is flawless, but there is a severe business-logic flaw. Find the flaw and document the required review comment.
12. **PR Narration Generation (Session 16):** Write an auditable "PR Narration" summarizing exactly *why* an AI agent chose a specific database index pattern, ensuring the justification meets strict enterprise compliance and audit standards.

Reading List

The following reading list is curated to provide credible, enterprise-grade context, completely avoiding generic AI hype.

Core Required

- **Vibe Coding is Not the Same as AI-Assisted Engineering** by Addy Osmani (2025). *Why it matters:* Establishes the foundational dichotomy of the course. It forcefully argues that unstructured momentum collapses under production demands, making it essential reading for Session 1.[6]
- **From Vibe Coding to Spec-Driven Development: How to Ship with AI at Scale** by Cloud2Labs (2026). *Why it matters:* Details the critical transition from ad-hoc prompts to structured specifications, highlighting the scalability issues of pure vibe coding. Required for Session 5.[15]
- **Claude Code Documentation: Plan Mode vs Act Mode** by Anthropic. *Why it matters:* The definitive, technical guide on utilizing read-only exploration safely, forming the basis

of the course's execution safety protocols. Required for Session 6.[20]

Strongly Recommended

- **The New SDLC: A Practical Guide to Agentic Engineering** by Alex Lavaee (2026). *Why it matters:* Explains precisely how the traditional Software Development Life Cycle is mathematically changing due to agentic integration.[42]
- **Comprehension Debt: The Hidden Cost of AI-Generated Code** by Addy Osmani (2026). *Why it matters:* Crucial for understanding why human review and architectural ownership must scale alongside AI generation.[3]
- **The Complete Guide to Claude Code V2: Global CLAUDE.md** by Community Contributors (2026). *Why it matters:* Provides battle-tested best practices for enforcing project-level guardrails and preventing context bloat.[33]

Claude Code / CLI Agent Specific

- **Claude Code Agent Teams Documentation** by Anthropic (2026). *Why it matters:* Essential, highly technical reading for orchestrating Opus 4.6 parallel agents and managing shared mailboxes.[16]
- **A Deep Dive Into MCP and the Future of AI Tooling** by Yoko Li / a16z (2025). *Why it matters:* The definitive, industry-standard explanation of why the Model Context Protocol is revolutionizing and standardizing enterprise integrations.[8]
- **Inside Claude Code: When to Use Plan Mode vs Skills vs Subagents** by Mustafa Zeeshan (2026). *Why it matters:* Provides practical, actionable mental models for choosing the exact right CLI tool for specific engineering tasks.[36]

Security / Prompt Injection / Safety

- **OWASP Top 10 for Agentic Applications** by OWASP GenAI Security (2026). *Why it matters:* The absolute industry standard for identifying execution-layer threats, shifting focus from content filtering to action verification. Required for Session 12.[21]
- **AI Safety and Security in 2026: The Urgent Need for Enterprise Governance** by Cranium.ai (2026). *Why it matters:* Connects deep technical AI risks directly to boardroom accountability and compliance.[43]
- **State of AI Agent Security 2026 Report** by Gravitee.io (2026). *Why it matters:* Provides hard, statistical data on the dangerous gap between rapid AI adoption and lagging security controls.[44]

Optional Advanced / Enterprise Workflow

- **NIST AI Agent Standards Initiative: Interoperable and Secure** by NIST CAISI (2026). *Why it matters:* Outlines the incoming federal and international standards for agentic interoperability and auditability.[39]
- **Code Execution with MCP: Building More Efficient Agents** by Anthropic Engineering

(2025). *Why it matters:* A deep technical dive into token consumption, context overload, and scaling tool usage efficiently.[45]

Assessment and Completion

To successfully exit the Foundations phase and be officially declared **"Ready for Labs,"** learners must pass a multi-tiered assessment designed to rigorously test methodology, CLI technical proficiency, and safety judgment.

1. Knowledge Checks (Automated)

Learners will complete a timed, 20-question assessment testing core definitions and architectural concepts. Topics include Comprehension Debt, SDD methodologies, MCP architecture, Context Rot mathematics, and the OWASP Agentic Top 10 vulnerabilities. *Pass condition: 85% or higher.*

2. Workflow Judgment Checks (Scenario-Based)

Learners are presented with three complex, failing AI scenarios. For example: an agent stuck in an infinite retry loop; an agent attempting to drop a production database table; an agent producing 10,000 lines of technically correct but unreviewable code. Learners must correctly identify the missing architectural guardrail (e.g., Missing SDD, missing CLAUDE.md constraint, improper Plan Mode usage) and prescribe the fix.

3. Safety Gates (The Hard Stops)

Learners must actively demonstrate the ability to configure a Zero Trust environment directly in the CLI. They must submit a permissions.json file that successfully restricts a simulated agent to strictly read-only tools and completely blocks external internet access. *Pass condition: 100% adherence to least-privilege principles. No exceptions are permitted.*

4. Artifact-Based Assessment (The Capstone)

Learners must submit a complete **Agentic Engineering Blueprint** based on a provided, complex mock-feature ticket. The blueprint must include:

1. A concise, 15-line Markdown Specification (SDD).
2. A CLAUDE.md rule set explicitly tailored to the feature's constraints.
3. An architectural decomposition mapping, breaking the feature into three distinct, isolated subagent tasks.
4. A PR Narration template explaining exactly how the AI's work will be audited by a human engineer.

Definition of "Ready for Labs":

A learner is officially "Ready for Labs" when they can interact with the Claude Code CLI fluently, understand exactly how to restrict its permissions securely, categorically refuse to prompt without a written specification, and treat all AI output as fundamentally untrusted material

requiring rigorous, structural human review.

Final Recommendation

Recommended Final Course Title:

AI-Accelerated Software Development: Structured Foundations

(Note: This title satisfies enterprise conservatism and procurement risk-aversion, while allowing the internal curriculum to aggressively train "Agentic Engineering" patterns.)

Recommended Module Structure:

1. The Agentic Paradigm & CLI Environment
2. Spec-Driven Development & Context Control
3. Orchestration, MCP & Subagents
4. Enterprise Security, Governance & Auditability
5. Applied Workflows & Lab Readiness

The 5 Biggest Improvements from the Redesign:

1. **Eradication of "Vibe Coding":** The formal, structural introduction of Spec-Driven Development (SDD) forces strict engineering discipline back into the AI workflow, ensuring accountability.
2. **CLI-Native Ecosystem:** Standardizing instruction entirely on the Claude Code CLI ensures learners build highly operational muscle memory, abandoning limited chatbot habits.
3. **Enterprise Standard Alignment:** Deep, technical integration of 2026 standards like the Model Context Protocol (MCP) and OWASP Agentic Top 10 ensures the curriculum remains cutting-edge and relevant.
4. **Operational Security Focus:** Transitioning the security module from abstract "AI ethics" to concrete Zero Trust permissioning and execution-layer threat modeling protects the enterprise.
5. **Direct Lab Alignment:** The final module perfectly simulates the exact reconnaissance and CI/CD triage motions required on Day 1 of the Applied Labs, eliminating onboarding friction.

The 5 Biggest Risks or Trade-offs:

1. **Steep Learning Curve:** Moving straight into CLI architecture, JSON-RPC (MCP), and Hooks may initially overwhelm engineers who are accustomed only to basic Copilot autocomplete functions.
2. **Tool Volatility:** The Claude Code CLI (especially features like Opus 4.6 Agent Teams and MCP integration) is evolving rapidly; the curriculum will require strict quarterly maintenance to prevent code rot.
3. **Token Cost:** Running Agent Teams and executing heavy Plan Mode reconnaissance during the exercises will consume significant API credits and tokens, raising the cost of

training delivery.

4. **Resistance to SDD:** Developers addicted to the instant gratification of "vibe coding" may actively resist the friction of having to write markdown specifications before generating code.
5. **Environment Setup Friction:** Securing native CLI installations and authenticating custom MCP servers within locked-down corporate VPNs or restricted networks frequently causes Day 1 training delays.

Works cited

1. From Vibe Coding to Vibe Engineering: Best Practices from the Front Lines | by Evan Rose, accessed April 5, 2026, https://medium.com/rose-digital/from-vibe-coding-to-vibe-engineering-best-practices-from-the-front-lines-1eb17b6f7cd4
2. Vibe Coding is a lie. Professional AI Development is just high-speed Requirements Engineering. : r/vibecoding - Reddit, accessed April 5, 2026, https://www.reddit.com/r/vibecoding/comments/1r0urgs/vibe_coding_is_a_lie_professional_ai_development/
3. Comprehension Debt — the hidden cost of AI generated code. | by Addy Osmani - Medium, accessed April 5, 2026, https://medium.com/@addyosmani/comprehension-debt-the-hidden-cost-of-ai-generated-code-285a25dac57e
4. Agentic Engineering - AddyOsmani.com, accessed April 5, 2026, https://addyosmani.com/blog/agentic-engineering/
5. From Copilots to Agentic Workflows: How AI in R&D Changes in 2026 - Causaly, accessed April 5, 2026, https://www.causaly.com/blog/from-copilots-to-agentic-workflows-how-ai-in-r-d-changes-in-2026
6. Vibe coding is not the same as AI-Assisted engineering. | by Addy Osmani - Medium, accessed April 5, 2026, https://medium.com/@addyosmani/vibe-coding-is-not-the-same-as-ai-assisted-engineering-3f81088d5b98
7. A Framework for Disciplined AI-Assisted Development - Ocula Technologies, accessed April 5, 2026, https://ocula.tech/resources/spec-driven-ai-collaboration
8. A Deep Dive Into MCP and the Future of AI Tooling | Andreessen Horowitz, accessed April 5, 2026, https://a16z.com/a-deep-dive-into-mcp-and-the-future-of-ai-tooling/
9. AI Engineer Roadmap 2026: Skills, Tools, and Career Path - Let's Data Science, accessed April 5, 2026, https://letsdatascience.com/blog/ai-engineer-roadmap-2026-skills-tools-and-career-path
10. Beyond Vibe-Coding: A Disciplined Workflow for AI-Assisted Software Development with Claude Code - #InnoBlog, accessed April 5, 2026, https://blog.innogames.com/beyond-vibe-coding-a-disciplined-workflow-for-ai-assisted-software-development-with-claude-code/

11. 10 Must-Have Skills for Claude (and Any Coding Agent) in 2026 | by unicodeveloper | Mar, 2026, accessed April 5, 2026, https://medium.com/@unicodeveloper/10-must-have-skills-for-claude-and-any-coding-agent-in-2026-b5451b013051
12. Agentic Engineering vs. Vibe Coding - Turing College, accessed April 5, 2026, https://www.turingcollege.com/blog/agentic-engineering-vs-vibe-coding
13. It's not vibe coding: Agentic engineering - Michael Kennedy, accessed April 5, 2026, https://mkennedy.codes/posts/its-not-vibe-coding-agentic-engineering/
14. Using spec-driven development with Claude Code | by Heeki Park ..., accessed April 5, 2026, https://heeki.medium.com/using-spec-driven-development-with-claude-code-4a1ebe5d9f29
15. From Vibe Coding to Spec-Driven AI Development at Scale - Cloud2 Labs, accessed April 5, 2026, https://cloud2labs.com/from-vibe-coding-to-spec-driven-development-how-to-ship-with-ai-at-scale/
16. Orchestrate teams of Claude Code sessions, accessed April 5, 2026, https://code.claude.com/docs/en/agent-teams
17. Model Context Protocol: MCP Implementation Guide Enterprise | Fortune 500 Adoption 2026, accessed April 5, 2026, https://www.synvestable.com/model-context-protocol.html
18. Disruptive Innovation or Industry Buzz? Understanding Model Context Protocol's Role in Data-Driven Agentic AI | Informatica, accessed April 5, 2026, https://www.informatica.com/blogs/disruptive-innovation-or-industry-buzz-understanding-model-context-protocols-role-in-data-driven-agentic-ai.html
19. Claude Code Setup Guide - Research Memex, accessed April 5, 2026, https://research-memex.org/implementation/agentic-ai-tools/claude-code-setup-guide
20. Plan Mode in Claude Code - Think Before You Build with AI - codewithmukesh, accessed April 5, 2026, https://codewithmukesh.com/blog/plan-mode-claude-code/
21. OWASP Top 10 for Agentic Applications for 2026, accessed April 5, 2026, https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
22. Agentic AI Security: Concerns and Best Practices in 2026 - Aisera, accessed April 5, 2026, https://aisera.com/blog/agentic-ai-security/
23. Agentic Engineering vs Vibe Coding — not the same thing : r/vibecoding - Reddit, accessed April 5, 2026, https://www.reddit.com/r/vibecoding/comments/1rk8bdl/agentic_engineering_vs_vibe_coding_not_the_same/
24. The Reality Behind the Buzz: The Current State of Agentic Engineering in 2025 | David Lozzi, accessed April 5, 2026, https://davidlozzi.com/2025/08/20/the-reality-behind-the-buzz-the-current-state-of-agentic-engineering-in-2025/
25. Addressing the OWASP Top 10 Risks in Agentic AI with Microsoft Copilot Studio,

accessed April 5, 2026, https://www.microsoft.com/en-us/security/blog/2026/03/30/addressing-the-owasp-top-10-risks-in-agentic-ai-with-microsoft-copilot-studio/
26. AI Adoption & Strategy - Baytech Consulting, accessed April 5, 2026, https://www.baytechconsulting.com/blog/category/ai-adoption-strategy
27. The Era of Vibe Coding: Radical Abstraction & The Agentic Architect | by Jasleen - Medium, accessed April 5, 2026, https://medium.com/data-science-collective/the-era-of-vibe-coding-radical-abstraction-the-agentic-architect-b1905f0acf2b
28. AI in SDLC: How to Use AI-Powered Software Development in 2026 - GoGloby, accessed April 5, 2026, https://gogloby.com/insights/ai-in-sdlc/
29. AI-Assisted Software Engineering: A Special Topics Course That Could Become Mainstream | NJIT News, accessed April 5, 2026, https://news.njit.edu/ai-assisted-software-engineering-special-topics-course-could-become-mainstream
30. AI-Assisted Coding and Generative Engineering | BCG X, accessed April 5, 2026, https://www.bcg.com/x/the-multiplier/ai-assisted-coding-generative-engineering
31. Claude Code in March 2026: The Economics of the Quota | by William Couturier - Medium, accessed April 5, 2026, https://medium.com/@william.couturier/claude-code-in-march-2026-the-economics-of-the-quota-792449b63edb
32. Claude Code CLI: The Definitive Technical Reference | Introl Blog, accessed April 5, 2026, https://introl.com/blog/claude-code-cli-comprehensive-guide-2025
33. The Complete Guide to Claude Code V2: CLAUDE.md, MCP, Commands, Skills & Hooks — Updated Based on Your Feedback : r/ClaudeAI - Reddit, accessed April 5, 2026, https://www.reddit.com/r/ClaudeAI/comments/1qcwckg/the_complete_guide_to_claude_code_v2_claudemd_mcp/
34. Claude Code Beyond Sub-Agents: Orchestrating Peer-to-Peer AI with Agent Teams | by kumaran srinivasan | Feb, 2026 | Medium, accessed April 5, 2026, https://medium.com/@kumaran.isk/claude-code-beyond-sub-agents-orchestrating-peer-to-peer-ai-with-agent-teams-3406d2169bfd
35. Claude Code CLI: The Complete Guide - Blake Crosley, accessed April 5, 2026, https://blakecrosley.com/guides/claude-code
36. Inside Claude Code: When to Use Plan Mode vs Skills vs Subagents in Real Playwright Automation | by Zeeshan Mustafa | Mar, 2026 | Medium, accessed April 5, 2026, https://medium.com/@mustafazeeshan333/inside-claude-code-when-to-use-plan-mode-vs-skills-vs-subagents-in-real-playwright-automation-136f6618b6c6
37. The Discipline Layer: Harnesses as the Missing Piece in Autonomous Coding - Maxim AI, accessed April 5, 2026, https://www.getmaxim.ai/blog/the-discipline-layer-harnesses-as-the-missing-piece-in-autonomous-coding/
38. GitHub - PaulDuvall/ai-development-patterns: A comprehensive collection of AI development patterns for building software with AI assistance, organized by

implementation maturity and development lifecycle phases. Includes Foundation, Development, and Operations patterns with practical examples and anti-patterns., accessed April 5, 2026, https://github.com/PaulDuvall/ai-development-patterns
39. Announcing the "AI Agent Standards Initiative" for Interoperable and Secure Innovation, accessed April 5, 2026, https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure
40. AI SDLC - How AI Is Rewriting Software Development | IdeaPlan, accessed April 5, 2026, https://www.ideaplan.io/ai-sdlc
41. The new AI-driven SDLC - CircleCI, accessed April 5, 2026, https://circleci.com/blog/ai-sdlc/
42. BCW 31: AI-enabled Company - Business Case Weekly - Substack, accessed April 5, 2026, https://businesscaseweekly.substack.com/p/bcw-31-ai-enabled-company
43. AI Security in 2026: Enterprise Governance, Risks, and Best Practices - Cranium AI, accessed April 5, 2026, https://cranium.ai/resources/blog/ai-safety-and-security-in-2026-the-urgent-need-for-enterprise-cybersecurity-governance/
44. State of AI Agent Security 2026 Report: When Adoption Outpaces Control - Gravitee, accessed April 5, 2026, https://www.gravitee.io/blog/state-of-ai-agent-security-2026-report-when-adoption-outpaces-control
45. Code execution with MCP: building more efficient AI agents - Anthropic, accessed April 5, 2026, https://www.anthropic.com/engineering/code-execution-with-mcp